

MINISTRY OF EDUCATION, CULTURE AND RESEARCH OF THE REPUBLIC OF MOLDOVA

Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software and Automation Engineering

Iulian Pavlov, FAF-241

Report

Laboratory work nr. 11

The Race and the Semaphore

Course: Operating Systems (Track A: the real OS, from the outside (Linux))

Checked by:
Reiţman Patricia, *University Assistant*
Department of Software and Automation Engineering,
FCIM Faculty, UTM

Chişinău – 2026

Contents

1 Theoretical Background and Objectives **1**
1.1 Laboratory Objectives and Scope 1
1.2 Three Synchronization Roles of a Semaphore 1

2 Part 1: Observing the Data Race (`race.c`) **2**
2.1 Implementation and Unprotected Runs 2
2.2 Part 1 Observation Analysis 2

3 Part 2: Fixing the Race (`mutex.c`, `sem.c`, `sem2.c`) **3**
3.1 Protecting the Critical Section 3
3.2 Performance Comparison 3
3.3 Part 2 Observation Analysis 4

4 Part 3: Semaphore as an Ordering Signal (`order.c`, `order1.c`) **4**
4.1 Coordinating Writer and Reader Threads 4
4.2 Part 3 Observation Analysis 5

5 Part 4: Semaphore as a Counter (`parking.c`) **5**
5.1 Bounded Concurrency with `PLACES = 3` and `PLACES = 1` 5
5.2 Part 4 Observation Analysis 6

6 Optional Extension: Lock-Free Atomics (`atomic.c`) **6**

7 Conclusions **7**

1 Theoretical Background and Objectives

1.1 Laboratory Objectives and Scope

Concurrent threads inside the same Linux process share one virtual address space, including global variables. Whenever multiple threads read and modify a shared variable without synchronization, preemptive scheduling and multicore execution interleave their machine instructions unpredictably, silently losing updates—a classic **data race**.

The objectives of Laboratory Work No. 11 are:

- **Observing a Data Race (`race.c`)**: Compiling without `-O2` (so increments stay memory-backed) and showing how two threads adding `1` one million times each fall short of `2 000 000`.
- **Mutual Exclusion (`mutex.c`, `sem.c`, `sem2.c`)**: Protecting the critical section with a POSIX mutex (`pthread_mutex_t`) and with a POSIX semaphore initialized to `1` (`sem_t`), verifying what breaks at initial count `2`, and profiling lock overhead with `time`.
- **Thread Signaling (`order.c`, `order1.c`)**: Enforcing strict writer-before-reader ordering using a semaphore initialized to `0`, and testing why starting at `1` causes premature reads.
- **Resource Counting (`parking.c`, `parking1.c`)**: Capping `6` car threads to `3` parking spots with a counting semaphore, then setting `PLACES` to `1` to verify mutex-like behavior.
- **Lock-Free Atomics (`atomic.c` – Optional)**: Testing hardware `atomic_fetch_add()` (`<stdatomic.h>`) against kernel-assisted locks.

1.2 Three Synchronization Roles of a Semaphore

A POSIX semaphore manages a non-negative integer counter via two atomic primitives: `sem_wait()` (decrements by `1`, blocking if currently `0`) and `sem_post()` (increments by `1`, waking a waiting thread). Varying only the initial value switches the semaphore across three distinct synchronization idioms summarized in Table 1.

Table 1: Semaphore Initial Values and Their Synchronization Semantics in Lab 11

Init	Role	Synchronization Meaning	Programs
`1`	Binary lock (mutex)	At most one thread executes inside the critical section at any time	`sem.c`, `parking1.c`
`0`	Ordering signal	A waiting thread blocks at `sem_wait()` until another calls `sem_post()`	`order.c`
`N` (`3`)	Counting semaphore	Bounds concurrent occupancy to at most `N` threads simultaneously	`parking.c`

2 Part 1: Observing the Data Race (`race.c`)

2.1 Implementation and Unprotected Runs

In `race.c`, two POSIX threads execute `add()`, each looping `N = 1 000 000` times to increment global `counter`. Compiling without `-O2` prevents the compiler from caching `counter` in a local register across loop iterations. *(Note: an earlier paste escape sequence in `notes.md` left `race.c` empty; the file was restored directly from the Lab 11 specification, compiled cleanly with `gcc -Wall -pthread`, and verified across multiple live executions.)*

Listing 1: Unprotected shared counter in `race.c`

```c
#include <stdio.h>
#include <pthread.h>
#define N 1000000
long counter = 0;
void *add(void *arg) {
    for (int i = 0; i < N; i++)
        counter++;
    return NULL;
}
int main(void) {
    pthread_t a, b;
    pthread_create(&a, NULL, add, NULL);
    pthread_create(&b, NULL, add, NULL);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("counter = %ld (expected %d)\n", counter, 2 * N);
    return 0;
}
```

```
$ gcc -Wall -pthread -o race race.c
$ ./race
counter = 1213503 (expected 2000000)
$ ./race
counter = 1073171 (expected 2000000)
$ ./race
counter = 1220817 (expected 2000000)
$ time ./race
counter = 1089313 (expected 2000000)

real  0m0.019s
user  0m0.025s
sys 0m0.003s
```

2.2 Part 1 Observation Analysis

1. **Three CPU steps behind `counter++`**: Although written as one C expression, `counter++` executes in hardware as three separate instructions: **(1) Load** the current integer from memory into a register; **(2) Add 1** inside the CPU ALU; and **(3) Store** the new register value back into `counter` in memory.

2. **How two threads lose an addition**: Assume `counter` holds `100`. Thread `a` loads `100` and computes `101`, then gets preempted (or runs concurrently on a second core) before storing. Thread `b` also loads the stale `100`, computes `101`, and writes `101`.

When Thread a resumes and writes its 101, both increments happened, yet memory holds 101 instead of 102.

3. **Why each run produces a different total**: Timer interrupts, scheduler decisions across cores, and memory-cache timing differ slightly between runs, shifting how many read–modify–write windows collide (1 213 503, 1 073 171, 1 220 817).

3 Part 2: Fixing the Race (mutex.c, sem.c, sem2.c)

3.1 Protecting the Critical Section

Two synchronization mechanisms serialize access to counter++:

- **POSIX Mutex (mutex.c)**: Uses a mutex statically initialized via PTHREAD_MUTEX_INITIALIZER to protect counter++ with lock/unlock calls.
- **POSIX Semaphore (sem.c)**: Uses sem_init(&s, 0, 1), wrapping counter++ between sem_wait(&s) and sem_post(&s).

Listing 2: Semaphore lock worker in sem.c (sem_init(&s, 0, 1))

```
sem_t s;
void *add(void *arg) {
    for (int i = 0; i < N; i++) {
        sem_wait(&s);
        counter++;
        sem_post(&s);
    }
    return NULL;
}
```

We also created sem2.c by setting sem_init(&s, 0, 2) (starting value 2) to test what happens when multiple permits are available simultaneously:

```
$ ./mutex && ./mutex && ./mutex
counter = 2000000 (expected 2000000)
counter = 2000000 (expected 2000000)
counter = 2000000 (expected 2000000)

$ ./sem && ./sem && ./sem
counter = 2000000 (expected 2000000)
counter = 2000000 (expected 2000000)
counter = 2000000 (expected 2000000)

$ sed 's/sem_init(&s, 0, 1)/sem_init(\&s, 0, 2)/' sem.c > sem2.c
$ gcc -Wall -pthread -o sem2 sem2.c && ./sem2 && ./sem2 && ./sem2
counter = 1629819 (expected 2000000)
counter = 1392310 (expected 2000000)
counter = 1598883 (expected 2000000)
```

3.2 Performance Comparison

Table 2 gathers live wall-clock, user-mode, and kernel-mode timings collected with time (including the optional lock-free ./atomic run from Section 6). In the earlier notes.md

log, `./mutex` took `0.197 s` and `./sem` took `0.235 s`; our fresh verification measured `0.195 s` and `0.282 s`, preserving the exact same performance ordering (`race` < `atomic` < `mutex` < `sem`).

Table 2: Measured Execution Time and Accuracy Across Synchronization Methods

Program	Mechanism	Result	real (s)	user (s)	sys (s)
`./race`	Unsynchronized	`1089313` (wrong)	0.019	0.025	0.003
`./atomic`	`atomic_fetch_add`	`2000000` (exact)	0.044	0.084	0.000
`./mutex`	`pthread_mutex_t`	`2000000` (exact)	0.195	0.185	0.192
`./sem`	`sem_t` (`init=1`)	`2000000` (exact)	0.282	0.260	0.295

3.3 Part 2 Observation Analysis

1. **Consistency of `mutex` and `sem`**: Running both binaries three times each (plus timed runs) always produced `counter = 2000000` (`expected 2000000`), proving mutual exclusion prevented lost updates.

2. **Behavior of `sem_init(&s, 0, 2)` (`sem2.c`)**: All runs failed (`1629819`, `1392310`, `1598883`; earlier `notes.md` runs gave `1610383`, `1914526`, `1386239`). Starting at 2 lets two threads pass `sem_wait(&s)` concurrently (`2 -> 1 -> 0`), allowing both threads inside the critical section and re-creating the race.

3. **Why the locked versions are slower**: Performing 2 000 000 lock acquisitions and releases introduces atomic compare-and-swap contention between CPU cores and kernel `futex` syscalls whenever a thread blocks or wakes (`sys` increases from `0.003 s` in `race` to `0.192 s` in `mutex` and `0.295 s` in `sem`).

4 Part 3: Semaphore as an Ordering Signal (`order.c`, `order1.c`)

4.1 Coordinating Writer and Reader Threads

In `order.c`, `main()` starts `reader` (thread `b`) before `writer` (thread `a`). Because `ready` is initialized via `sem_init(&ready, 0, 0)`, `reader` blocks on `sem_wait(&ready)` after printing its first line until `writer` sleeps 1 s, writes `"hello"` into `message`, and calls `sem_post(&ready)`. Variant `order1.c` changes the start count to `1`.

Listing 3: Writer-before-reader coordination in `order.c`

```c
char message[32];
sem_t ready;
void *writer(void *arg) {
    sleep(1);
    strcpy(message, "hello");
    printf("A: message written\n");
    sem_post(&ready);
    return NULL;
}
void *reader(void *arg) {
```

```c
    printf("B: waiting for the message...\n");
    sem_wait(&ready);
    printf("B: I read \"%s\"\n", message);
    return NULL;
}
```

```
$ ./order
B: waiting for the message...
A: message written
B: I read "hello"

$ ./order1
B: waiting for the message...
B: I read ""
A: message written
```

4.2 Part 3 Observation Analysis

1. **Effect of `sem_init(&ready, 0, 1)` (`order1.c`)**: `reader` immediately prints `B: I read ""` (an empty string) before `writer` finishes sleeping and prints `A: message written`. With an initial value of `1`, `sem_wait(&ready)` immediately consumes the spare token (`1 -> 0`) instead of waiting, reading the zero-initialized global array `message` before `strcpy()` runs.

2. **Meaning of a semaphore initialized to `0`**: A semaphore starting at `0` represents a signal ("nothing is ready yet") on which a consumer blocks at `sem_wait()` until another thread signals completion with `sem_post()`.

5 Part 4: Semaphore as a Counter (`parking.c`)

5.1 Bounded Concurrency with `PLACES = 3` and `PLACES = 1`

In `parking.c`, `CARS = 6` threads compete for `PLACES = 3` spots tracked by `places`. The semaphore limits capacity via `sem_wait()` on entry and `sem_post()` after `sleep(1)`, whereas `lock` guards `inside++`, `inside--`, and `printf()`. Variant `parking1.c` sets `PLACES` to `1`.

Listing 4: Bounded parking simulation in `parking.c`

```c
#define PLACES 3
#define CARS 6
sem_t places;
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int inside = 0;

void *car(void *arg) {
    long id = (long)arg;
    sem_wait(&places);
    pthread_mutex_lock(&lock);
    inside++;
    printf("car %ld enters (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);
    sleep(1);
    pthread_mutex_lock(&lock);
```

```
    inside--;
    printf("car %ld leaves (inside: %d)\n", id, inside);
    pthread_mutex_unlock(&lock);
    sem_post(&places);
    return NULL;
}
```

```
$ ./parking
car 1 enters (inside: 1)
car 3 enters (inside: 2)
car 2 enters (inside: 3)
car 2 leaves (inside: 2)
car 3 leaves (inside: 1)
car 1 leaves (inside: 0)
car 5 enters (inside: 1)
car 4 enters (inside: 2)
car 6 enters (inside: 3)
car 5 leaves (inside: 2)
car 4 leaves (inside: 1)
car 6 leaves (inside: 0)

$ ./parking1
car 1 enters (inside: 1)
car 1 leaves (inside: 0)
car 2 enters (inside: 1)
car 2 leaves (inside: 0)
car 3 enters (inside: 1)
car 3 leaves (inside: 0)
car 4 enters (inside: 1)
car 4 leaves (inside: 0)
car 5 enters (inside: 1)
car 5 leaves (inside: 0)
car 6 enters (inside: 1)
car 6 leaves (inside: 0)
```

5.2 Part 4 Observation Analysis

1. **Maximum occupancy and enforcing code**: The largest observed value of `inside` is `3` (`car 2 enters (inside: 3)`, `car 6 enters (inside: 3)`). This invariant is guaranteed by `sem_init(&places, 0, PLACES);` (initializing 3 tokens) paired with `sem_wait(&places);` before incrementing `inside`, which blocks cars 4–6 until an active car releases a token via `sem_post(&places);`.

2. **Setting `PLACES` to 1 (`parking1.c`)**: Each car enters and exits one at a time (`inside` alternates between 1 and 0), serializing execution across 6 seconds. A counting semaphore initialized to 1 becomes a **binary semaphore (mutex)**.

6 Optional Extension: Lock-Free Atomics (`atomic.c`)

For a single counter, `atomic.c` replaces locks with C11 `<stdatomic.h>`: `atomic_long counter` initialized to 0 and updated via `atomic_fetch_add(&counter, 1)`. The compiler emits an indivisible CPU atomic add instruction, preventing interleaving without kernel blocking.

```
$ time ./atomic
counter = 2000000 (expected 2000000)

real  0m0.044s
user  0m0.084s
sys 0m0.000s
```

With `0.000 s` kernel time, `./atomic` achieved exact accuracy in `0.044 s`—roughly **4.4x faster** than `./mutex` (`0.195 s`) and **6.4x faster** than `./sem` (`0.282 s`).

7 Conclusions

1. **Root Cause of Data Races**: `counter++` comprises three non-atomic CPU steps (load, add, store); without mutual exclusion, concurrent threads overwrite each other's intermediate results (`1.07M-1.22M` instead of `2 000 000`).

2. **Versatility of POSIX Semaphores**: A single semaphore API (`sem_wait` and `sem_post`) covers three fundamental patterns purely through its initial count: `1` for mutual exclusion, `0` for execution-order signaling, and `N` for bounded resource concurrency.

3. **Correctness Requires Exact Initial Counts**: Starting a lock semaphore at `2` (`sem2.c`) admits two threads simultaneously and re-creates the race, while starting a signal semaphore at `1` (`order1.c`) lets the reader proceed before the writer produces data.

4. **Synchronization Performance Trade-Offs**: Locks enforce correctness at the price of atomic contention and kernel `futex` syscalls (`0.195 s` for `mutex`, `0.282 s` for `sem`), whereas lock-free hardware atomics (`atomic_fetch_add`) deliver exact results for single counters in `0.044 s` with zero kernel overhead.